Event ends in 2 days 2 hours 55 minutes.

AWS resource planning translates your architecture design into specific, configurable AWS resources. This section creates detailed specifications for all AWS services with exact configurations, policies, and best practices.

Exercise 1.1: S3 and CloudFront Specification

Create detailed specifications for static website hosting and content delivery:

~~~
Create comprehensive S3 and CloudFront specifications for our chatbot frontend:

**S3 Bucket Configuration**:
Use the AWS documentation MCP server to research current S3 best practices, then specify:

**Primary Frontend Bucket**:
- **Bucket Name**: `chatbot-frontend-${AWS::AccountId}-${AWS::Region}`
- **Purpose**: Host static website files (HTML, CSS, JS, assets)
- **Region**: Primary deployment region with cross-region replication options
- **Versioning**: Enabled with lifecycle policies for cost optimization
- **Encryption**: Server-side encryption with AES-256 (SSE-S3)
- **Public Access**: Blocked (access only via CloudFront OAI)
- **Lifecycle Rules**:
  - Delete non-current versions after 30 days
  - Transition to IA after 30 days for large assets
  - Delete incomplete multipart uploads after 7 days

**Bucket Policies**:
```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::cloudfront:user/CloudFront Origin Access Identity"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::chatbot-frontend-${AWS::AccountId}-${AWS::Region}/*"
    },
    {
      "Sid": "DenyDirectPublicAccess",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::chatbot-frontend-${AWS::AccountId}-${AWS::Region}",
        "arn:aws:s3:::chatbot-frontend-${AWS::AccountId}-${AWS::Region}/*"
      ],
      "Condition": {
        "StringNotEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::${AWS::AccountId}:distribution/*"
        }
      }
    }
  ]
}
```

**CloudFront Distribution Configuration**:
- **Origins**:
  - Primary: S3 bucket with Origin Access Identity (OAI)
  - API: API Gateway for backend communication
- **Behaviors**:
  - Default: S3 bucket (/, *.html, *.css, *.js, *.png, *.jpg, *.svg)
  - /api/*: API Gateway with no caching
  - /assets/*: S3 bucket with long-term caching (1 year)
- **Caching Strategy**:
  - HTML files: No caching (Cache-Control: no-cache)
  - CSS/JS files: 1 year caching with versioning
  - Images/assets: 1 year caching
  - API responses: No caching
- **Security Headers**:
  - Strict-Transport-Security: max-age=31536000; includeSubdomains; preload
  - Content-Security-Policy: default-src 'self'; connect-src 'self' https://*.execute-api.${AWS::Region}.amazonaws.com; script-src 'self'; style-src 'self' 'unsafe-inline'
  - X-Content-Type-Options: nosniff
  - X-Frame-Options: DENY
  - X-XSS-Protection: 1; mode=block
  - Referrer-Policy: strict-origin-when-cross-origin

Research current CloudFront best practices and security recommendations using the AWS documentation MCP server.
~~~

Expected Outcome

- Complete S3 bucket specification with security and lifecycle policies
- Detailed CloudFront configuration with caching and security headers
- Cost optimization strategies for storage and data transfer
- Security controls preventing direct S3 access
- Performance optimization through proper caching strategies

Exercise 1.2: API Gateway Specification

Create comprehensive API Gateway configuration for the chatbot backend:

~~~
Design detailed API Gateway specifications for our chatbot API:

**REST API Configuration**:
- **API Name**: `chatbot-api-${Environment}`
- **Description**: Serverless chatbot API with authentication and rate limiting
- **Endpoint Type**: Regional (for better performance and lower latency)
- **API Version**: v1 (with versioning strategy for future updates)

**Resource and Method Specifications**:

**Authentication Endpoints**:
- `POST /auth/register` - User registration
- `POST /auth/login` - User authentication
- `POST /auth/refresh` - Token refresh
- `POST /auth/logout` - User logout
- `GET /auth/profile` - User profile retrieval

**Chat Endpoints**:
- `POST /chat/message` - Send chat message
- `GET /chat/history` - Retrieve conversation history
- `DELETE /chat/history/{conversationId}` - Delete conversation
- `GET /chat/conversations` - List user conversations

**Utility Endpoints**:
- `GET /health` - Health check
- `GET /version` - API version information

**Request/Response Models**:
```json
{
  "ChatMessageRequest": {
    "type": "object",
    "required": ["message"],
    "properties": {
      "message": {
        "type": "string",
        "minLength": 1,
        "maxLength": 4000
      },
      "conversationId": {
        "type": "string",
        "pattern": "^[a-zA-Z0-9-_]{1,50}$"
      }
    }
  },
  "ChatMessageResponse": {
    "type": "object",
    "properties": {
      "response": {"type": "string"},
      "conversationId": {"type": "string"},
      "timestamp": {"type": "string", "format": "date-time"},
      "messageId": {"type": "string"}
    }
  }
}
```

**CORS Configuration**:
- **Allowed Origins**: CloudFront distribution domain only
- **Allowed Methods**: GET, POST, PUT, DELETE, OPTIONS
- **Allowed Headers**: Content-Type, Authorization, X-Api-Key, X-Requested-With
- **Exposed Headers**: X-Request-ID, X-Rate-Limit-Remaining
- **Allow Credentials**: true
- **Max Age**: 3600 seconds

**Throttling and Usage Plans**:
- **Default Throttling**: 1000 requests per second, 2000 burst
- **Per-User Throttling**: 100 requests per minute per API key
- **Usage Plans**:
  - Free Tier: 1000 requests per day
  - Premium Tier: 10000 requests per day
  - Enterprise Tier: Unlimited with custom throttling

Use the AWS documentation MCP server to research current API Gateway best practices and security configurations.
~~~

Expected Outcome

- Complete API Gateway specification with all endpoints and methods
- Detailed request/response models with validation
- Comprehensive CORS configuration for security
- Throttling and usage plans for cost control and abuse prevention
- Integration patterns with Lambda functions

Exercise 1.3: Lambda Functions Specification

Create detailed specifications for all Lambda functions:

~~~
Design comprehensive Lambda function specifications for our chatbot backend:

**Chat Processing Function**:
- **Function Name**: `chatbot-chat-processor-${Environment}`
- **Runtime**: Node.js 18.x (latest LTS)
- **Memory**: 512 MB (optimized for Bedrock API calls)
- **Timeout**: 30 seconds (sufficient for AI processing)
- **Architecture**: arm64 (for cost optimization)
- **Reserved Concurrency**: 100 (prevent cost overruns)
- **Environment Variables**:
  ```json
  {
    "BEDROCK_MODEL_ID": "anthropic.claude-3-haiku-20240307-v1:0",
    "BEDROCK_REGION": "${AWS::Region}",
    "LOG_LEVEL": "INFO",
    "MAX_TOKENS": "1000",
    "TEMPERATURE": "0.7",
    "CONVERSATION_TABLE": "${ConversationTableName}",
    "JWT_SECRET_ARN": "${JWTSecretArn}"
  }
  ```

**Authentication Function**:
- **Function Name**: `chatbot-auth-handler-${Environment}`
- **Runtime**: Node.js 18.x
- **Memory**: 256 MB (lightweight authentication processing)
- **Timeout**: 10 seconds
- **Architecture**: arm64
- **Reserved Concurrency**: 50
- **Environment Variables**:
  ```json
  {
    "USER_TABLE": "${UserTableName}",
    "JWT_SECRET_ARN": "${JWTSecretArn}",
    "JWT_EXPIRY": "3600",
    "BCRYPT_ROUNDS": "12",
    "LOG_LEVEL": "INFO"
  }
  ```

**User Management Function**:
- **Function Name**: `chatbot-user-manager-${Environment}`
- **Runtime**: Node.js 18.x
- **Memory**: 256 MB
- **Timeout**: 15 seconds
- **Architecture**: arm64
- **Reserved Concurrency**: 25
- **Environment Variables**:
  ```json
  {
    "USER_TABLE": "${UserTableName}",
    "LOG_LEVEL": "INFO",
    "ENCRYPTION_KEY_ARN": "${EncryptionKeyArn}"
  }
  ```

**Lambda Layer Specifications**:
- **Common Dependencies Layer**:
  - AWS SDK v3
  - JWT libraries
  - Validation libraries
  - Logging utilities
- **Bedrock Integration Layer**:
  - Bedrock SDK
  - AI model utilities
  - Response processing libraries

**Error Handling and Retry Configuration**:
- **Dead Letter Queues**: Configure for failed invocations
- **Retry Configuration**: Exponential backoff with jitter
- **Error Monitoring**: CloudWatch alarms for error rates
- **Circuit Breaker**: Implement for external service calls

Research current Lambda best practices including performance optimization, security, and cost management using the AWS documentation MCP server.
~~~

Expected Outcome

- Detailed Lambda function specifications with optimal configurations
- Environment variable management and security considerations
- Lambda layers for code reuse and dependency management
- Error handling and retry strategies for resilience
- Performance and cost optimization configurations

Exercise 1.4: IAM Roles and Policies Specification

Create comprehensive IAM specifications following least privilege principles:

~~~
Design detailed IAM roles and policies for our chatbot infrastructure:

**Lambda Execution Roles**:

**Chat Processor Role**:
```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:${AWS::Region}:${AWS::AccountId}:log-group:/aws/lambda/chatbot-chat-processor-*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel"
      ],
      "Resource": "arn:aws:bedrock:${AWS::Region}::foundation-model/anthropic.claude-3-haiku-20240307-v1:0"
    },
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb:UpdateItem",
        "dynamodb:Query"
      ],
      "Resource": "arn:aws:dynamodb:${AWS::Region}:${AWS::AccountId}:table/chatbot-conversations-${Environment}"
    },
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": "arn:aws:secretsmanager:${AWS::Region}:${AWS::AccountId}:secret:chatbot-jwt-secret-*"
    }
  ]
}
```

**Authentication Handler Role**:
```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:${AWS::Region}:${AWS::AccountId}:log-group:/aws/lambda/chatbot-auth-handler-*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb:UpdateItem"
      ],
      "Resource": "arn:aws:dynamodb:${AWS::Region}:${AWS::AccountId}:table/chatbot-users-${Environment}"
    },
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": "arn:aws:secretsmanager:${AWS::Region}:${AWS::AccountId}:secret:chatbot-jwt-secret-*"
    }
  ]
}
```

**API Gateway Execution Role**:
```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "lambda:InvokeFunction"
      ],
      "Resource": [
        "arn:aws:lambda:${AWS::Region}:${AWS::AccountId}:function:chatbot-*-${Environment}"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:${AWS::Region}:${AWS::AccountId}:log-group:/aws/apigateway/chatbot-api-*"
    }
  ]
}
```

**CloudFormation/CDK Deployment Role**:
- **Permissions**: Full access to create and manage all chatbot resources
- **Conditions**: Restrict to specific resource naming patterns
- **MFA Requirement**: Require MFA for production deployments
- **Time-based Access**: Limit deployment windows for production

**Cross-Account Access Policies** (if needed):
- **Development Account Access**: Limited permissions for developers
- **Production Account Access**: Restricted to deployment automation only
- **Audit Account Access**: Read-only access for compliance and monitoring

Research current IAM best practices and security recommendations using the AWS documentation MCP server, including permission boundaries and access analyzer.
~~~

Expected Outcome

- Comprehensive IAM roles following least privilege principles
- Detailed policies with specific resource restrictions
- Cross-account access patterns for multi-environment deployments
- Security controls including MFA and time-based access
- Compliance with AWS security best practices

Exercise 1.5: Data Storage and Bedrock Integration Specification

Create specifications for data storage and AI service integration:

~~~
Design comprehensive data storage and Bedrock integration specifications:

**DynamoDB Tables Specification**:

**Users Table**:
- **Table Name**: `chatbot-users-${Environment}`
- **Partition Key**: `userId` (String)
- **Attributes**:
  - `email` (String, GSI partition key)
  - `passwordHash` (String, encrypted)
  - `profile` (Map: firstName, lastName, preferences)
  - `createdAt` (String, ISO timestamp)
  - `lastLoginAt` (String, ISO timestamp)
  - `isActive` (Boolean)
- **Global Secondary Indexes**:
  - `EmailIndex`: Partition key `email`, for login lookups
- **Encryption**: Customer-managed KMS key
- **Point-in-time Recovery**: Enabled
- **Backup**: Daily automated backups with 30-day retention

**Conversations Table**:
- **Table Name**: `chatbot-conversations-${Environment}`
- **Partition Key**: `userId` (String)
- **Sort Key**: `conversationId` (String)
- **Attributes**:
  - `messages` (List of Maps: messageId, content, timestamp, role)
  - `createdAt` (String, ISO timestamp)
  - `updatedAt` (String, ISO timestamp)
  - `title` (String, conversation summary)
  - `isActive` (Boolean)
- **TTL**: `expiresAt` attribute for automatic cleanup
- **Encryption**: Customer-managed KMS key
- **Stream**: Enabled for real-time processing

**Amazon Bedrock Integration**:
- **Primary Model**: `anthropic.claude-3-haiku-20240307-v1:0`
- **Fallback Model**: `anthropic.claude-3-sonnet-20240229-v1:0`
- **Model Configuration**:
  ```json
  {
    "temperature": 0.7,
    "top_p": 0.9,
    "max_tokens": 1000,
    "stop_sequences": ["Human:", "Assistant:"]
  }
  ```
- **Cost Optimization**:
  - Request batching for multiple messages
  - Response caching for common queries
  - Model selection based on complexity
  - Usage monitoring and alerting
- **Error Handling**:
  - Retry logic with exponential backoff
  - Circuit breaker for service failures
  - Fallback to cached responses
  - Graceful degradation strategies

**Secrets Manager Configuration**:
- **JWT Secret**: Automatically rotated every 90 days
- **Database Encryption Keys**: Customer-managed KMS keys
- **API Keys**: Third-party service credentials
- **Certificates**: SSL/TLS certificates for custom domains

**S3 Additional Buckets** (if needed):
- **Logs Bucket**: CloudTrail and application logs
- **Backup Bucket**: Database and configuration backups
- **Assets Bucket**: User-uploaded files (if supported)

Research current DynamoDB best practices, Bedrock pricing optimization, and data management strategies using the AWS documentation MCP server.
~~~

Expected Outcome

- Complete data storage specifications with security and performance considerations
- Detailed Bedrock integration with cost optimization strategies
- Comprehensive secrets management and encryption configuration
- Data lifecycle and backup strategies
- Performance optimization for database operations

Key Deliverables

After completing the AWS Resource Planning exercises, you should have:

1. **Complete S3 and CloudFront Specifications**: Static hosting with security and performance optimization
2. **Detailed API Gateway Configuration**: RESTful API with authentication, validation, and rate limiting
3. **Comprehensive Lambda Function Specifications**: Optimized serverless compute with proper error handling
4. **IAM Roles and Policies**: Least privilege security with comprehensive access controls
5. **Data Storage and AI Integration**: Scalable data management with cost-optimized Bedrock integration

Integration with Advanced Features

Your AWS resource planning leverages:

- **AWS Documentation MCP Server**: For current best practices and service configurations
- **Agent Steering**: For applying AWS architectural and security standards
- **Agent Hooks**: For validating resource specifications and updating documentation

Next Steps

With detailed AWS resource specifications complete, you're ready to design the security infrastructure that will protect your chatbot application. The next section will focus on comprehensive security controls, compliance requirements, and threat mitigation strategies that build upon your resource specifications.

Previous Next